

1 Tutorial: Using Node.js to Call Services

1.1 Objective of the tutorial

This tutorial illustrates how to call system services using JavaScript in Node.js.

1.2 Requirements

- To proceed with the tutorial, you need access to an installed system. Therefore, you need the credentials.
- The JavaScript runtime environment Node.js (tested with version 8.9.4) must be installed on the local computer.

1.3 Installation

The source code is available support portal to download. It is located in the ZIP file in the subdirectory "tutorial_service_call_javascript".

The source code is also included in the MIP-SDK as a separate ZIP file.

Unpack the ZIP file or the sub directory " „tutorial_service_call_javascript“ aus der ZIP-Datei in ein Verzeichnis Ihrer Wahl (z. B. C:\Users\<username>\Tutorial).

1.4 Operation and result

As part of the tutorial, the sample program carries out the following steps:

1. Add a unit with typo (parameter *units.designation* is "pice" instead of "piece") using the service MDUnits.insert
 - Display the created unit using the service MDUnits.list
2. Correct the spelling mistake using the service MDUnits.update
 - Display the created unit using the service MDUnits.list
3. Delete the unit using the service MDUnits.delete
 - Display the created unit using the service MDUnits.list
4. Add a unit where a mandatory parameter is missing (*units.classification*) using the service MDUnits.insert

Change the connection data to the server to include your system. The file "settings.json" of the directory where you copied the contents of the ZIP archive includes the connection data. For example: "C:\Users\<UserName>\Tutorial\settings.json". The contents of the file *settings.json* are described [here](#).

Open a command line on the local computer to run the sample program. In the command line, go to the directory where you unpacked the source code of the tutorial. Execute the sample program:

Command	\$TUTORIALINSTALLATIONDIR\$/npm start
---------	---------------------------------------

The output generated while executing the sample program shows the single steps and their result.

```
PS C:\Projects\Tutorials\tutorial_service_call_javascript> npm start
> tutorial-service-call-javascript@1.0.0 start C:\Projects\Tutorials\tutorial_service_call_javascript
> node index.js

Step 1: insert an unit with a typo
insertUnit succeeded
List the unit
listUnits succeeded
Print units:
units.unit          | units.modified_by | units.unitiso      | units.designation  | units.classification | units.modified_ts   | units.si_unit      |
PC                  | 12345             | PC                 | piece              | Pieces               | 2018-04-24T13:57:49+02:00 | null               |

Step 2: update the unit to correct the typo
updateUnit succeeded
List the unit
listUnits succeeded
Print units:
units.unit          | units.modified_by | units.unitiso      | units.designation  | units.classification | units.modified_ts   | units.si_unit      |
PC                  | 12345             | PC                 | piece              | Pieces               | 2018-04-24T13:57:53+02:00 | null               |

Step 3: delete the unit
deleteUnit succeeded
List the unit
listUnits succeeded
No record available
Step 4: insert an unit with a missing mandatory parameter
insertUnit: Not null value is null for parameter units.classification
PS C:\Projects\Tutorials\tutorial_service_call_javascript>
```

1.5 Https

You can use the secure transfer protocol https to operate the sample program. To run the example program with https you have to put the file "ca-root.pem" into the directory "\$TUTORIALINSTALLATIONDIR\$" of the tutorial. The "ca-root.pem" file contains the root certificate.

If you want to run the example program with https, open a command line as described in 1.4. Execute the sample program at this point:

Command	\$TUTORIALINSTALLATIONDIR\$/npm run start_https
---------	---

1.6 Source code description of the sample program

The sample program mainly consists of two modules:

- util
- editUnits

A new unit is added, changed, displayed and deleted in the installed sample program (section 1.3 describes how to install the program). Additionally, an attempt is made to add an erroneous unit.

1.6.1 Util

The *util* module includes the following functions:

- Defining the access ID (constant: `access_ID`)
- Loading http settings from a JSON file (`loadHttpSettings`)
- Getting the used http settings (`getHttpSettings`)
- Processing the callback of requests (`requestCallback`)
- Displaying data after calling a list service (`printData`)
- Sending a request (`sendRequest`)

1.6.1.1 Defining access ID

The access ID is defined as constant in the util module and transferred as header information when a request is sent to the server (`sendRequest`). If this access ID is missing, an error is issued.

Define the access ID in section "constants" in the upper part of the module.

```
...
//Constants
const tab = "    "; //Indent of the lines
const pad end = 25; //Width of the columns
const access_ID = "00012345"; //Access-ID
...
```

1.6.1.2 loadHttpSettings

The function `loadHttpSettings` imports the JSON file `settings.json`. The function uses the content of the JSON file to generate a JavaScript object. The file `settings.json` must be stored in the root directory of the program.

The contents of the file `settings.json` are described [here](#).

1.6.1.3 getHttpSettings

The function `getHttpSettings` is a getter for the settings object.

1.6.1.4 requestCallback

The sample program calls the function `requestCallback` after an http request. The function includes, among others, the response and a callback function as parameters. The job of this function is to parse the response. The method generates the command line output according to the result.

```
try {
  var ob = JSON.parse(data);
  var status = null;
  if(!Array.isArray(ob)) {
    console.log(ob);
    status = {"message": ob};
  }
  else if(ob.length === 0 || (typeof ob[1] !== "undefined" && ob[1]["__rowType"] === "DATA")) {
    console.log(tab + functionName + " succeeded");
  }
  else if(ob[0]["__rowType"] === "ERROR") {
    console.error(tab + functionName + ": " + ob[0].message);
    status = {"message": ob[0].data.messages[0].parameters[0]};
  }
  else {
    console.log(functionName + ": Something unexpected happened");
  }
}
```

```
    status = {"message": "Something unexpected happened"};
  }

  if(callCallback) {
    //Return the error code
    callback(status);
  }
}
catch(error) {
  console.error(functionName + ": " + error);
}
```

As indicated in the listing above, a check is made to identify whether the response is an array or an object. It is an http error if the response is an object.

If successful and depending on the request, the service returns either an empty array or an array with at least two elements. The attribute `__rowType` distinguishes the elements. The first element includes meta information (`__rowType` is "META"). The other elements include the requested data (`__rowType` is "DATA").

In case of error, an array including only one single element is returned (`__rowType` is "ERROR"). The object's attribute `message` includes the error message. You can find the error code in the attribute `data.messages[0].parameters[0]`.

The function parameter `callback` is a function. After evaluating the response, the function `requestCallback` calls the `callback` function. If successful, the argument is `null`. In case of an error, the transferred argument is an object with the attribute `message`.

1.6.1.5 printData

The function `printData` displays the response of a *list request* in the command line. The function expects as parameter the response array as JSON string. Data is displayed in a table. In the table view, the parameters of the *META* element provide the header and the *DATA* elements provide the rows.

1.6.1.6 sendRequest

The function `sendRequest` creates a *Post Request* and sends it to a server of the system. The parameter `parameters` includes the body of the request. The parameter has the type *String*. The parameter `path` is the path of the request (e.g. `/data/BOOrder/insert`).

The sample program sends an empty `cookie` header field with the first request. The service sends a cookie in the header of the response. The function `sendRequest` of the sample program reads this cookie from the header field `set-cookie` and stores it. The stored cookie is transferred along with the headers of the next requests. This prevents new sessions from being created on the server with every request.

The function `sendRequest` calls the callback, once a response has arrived.

```
var req = http.request(options, (res) => {
  //Save the cookie sent from server
  if(typeof res.headers["set-cookie"] !== "undefined") {
    cookie = res.headers["set-cookie"][0];
  }

  //Read the data
```

```
let data = "";
res.on("data", (result) => {
  data += result;
});

//Data is complete, call the callback
res.on("end", () => {
  if(typeof callback === "function") {
    callback(null, data);
  }
});
});
```

1.6.2 editUnits

The *editUnits* module includes functions to execute requests. To do so, the function of the *workplaceStatusEvaluation* module calls the *sendRequest* method of the *util* module. The functions prepare the request bodies. The functions transfer the request bodies along with the corresponding path and a callback.

The module includes the following functions:

- Create a new unit (*insertMDUnits*)
- Change a unit (*updateMDUnits*)
- Display units (*listMDUnits*)
- Delete a unit (*deleteMDUnits*)

All these functions have the same structure. See the following example of *deleteMDUnits*:

```
function deleteMDUnits(unit = {}, callback = ()=>{}) {
  //Check if fields exists
  if( typeof unit.classification === "undefined" ||
      typeof unit.unit === "undefined" ||
      typeof unit.unitiso === "undefined" ||
      typeof unit.designation === "undefined"
  ) {
    callback({ "message": "insertMDUnits: field undefined" });
    return;
  }
  //The parameters are sent as a JSON string
  var parameters = JSON.stringify({
    "params": [
      {
        "acronym": "units.unit",
        "value" : unit.unit,
        "operator" : "EQUAL"
      }
    ]
  });
  util.sendRequest(parameters, "/data/MDUnits/delete", callback);
}
```

The above listing shows that the function expects two parameters:

- *Unit*: an object
- *Callback*: a callback function

The function uses the parameter *unit* to generate a JSON string. The function transfers this string with the path *"/data/MDUnits/delete"* and the callback to the method *sendRequest* of the *util* module.

The functions *insertMDUnits*, *updateMDUnits*, *listMDUnits* and *deleteMDUnits* are encapsulated with wrappers (e. g. *insertUnit*) in the index file of the *editUnits* module (*/app/index.js*). For a sequential process, the previously processed wrapper receives an (anonymous) function as callback. This anonymous function calls the next wrapper. The following listing shows an example of how to create and display a unit.

```
insertUnit(unit, () => {  
  listUnit(unit, () => {});  
});
```